

Segment 4 — CHROM/POS Pulse Extraction & FFT Heart Rate

Spandan: Contactless Vital Sign Monitoring — EEE 312 DSP Project, BUET

Team Spandan

August 5, 2026

Contents

Why This Segment Exists	1
Why Raw-Green FFT Alone Is Motion-Fragile	2
What CHROM and POS Physically Do Differently — the Skin-Tone-Vector Picture	2
Why Re-Filtering After Combination Is Necessary	3
The FFT Window-Length vs. Frequency-Resolution Tradeoff, in Concrete bpm Terms	4
Common Pitfalls in This Segment	5
How This Segment’s Three Estimates Feed Into Segment 6	5
Verification on This Project’s Own Data	6

Why This Segment Exists

Segment 3 handed us three clean, bandpass-filtered signals — `R_filtered(t)`, `G_filtered(t)`, `B_filtered(t)` — each restricted to the 0.7-4 Hz pulse band, with drift and out-of-band noise already removed. The most obvious next step is to just pick one of them, say the green channel (green is known to carry the strongest plethysmographic signal of the three, since hemoglobin absorbs green light more strongly than red), run an FFT on it, and call the dominant frequency the heart rate.

That naive approach — call it **green-only** — is exactly what this segment implements as a baseline, alongside two much better alternatives: **CHROM** and **POS**, two published algorithms that combine all three color channels together instead of using just one. This document explains why combining channels helps, what re-filtering after combination is for, what the FFT peak-picking step actually promises (and doesn’t), and the concrete resolution limits of doing this on UBFC-rPPG-length clips.

Intuition First

Intuition first. Picking one channel and hoping it's clean enough is like trying to follow one radio station by ear in a room with several overlapping broadcasts, using only one antenna pointed in a fixed direction. CHROM and POS are like combining several antennas pointed in different directions and adding their signals together in a way that's specifically tuned to cancel the interference and reinforce the one broadcast you actually want — using more information (three channels instead of one) to do a strictly better job than any single channel could alone.

Why Raw-Green FFT Alone Is Motion-Fragile

Green-only FFT works reasonably well on a perfectly still subject under constant lighting — and on real UBFC-rPPG footage, it often does show a visible peak somewhere in the valid band. The problem is *robustness*, not correctness in the ideal case:

- **Motion moves the whole color point, not just the pulse.** Any small head movement, re-positioning, or facial expression change shifts the ROI's average brightness in ways that have nothing to do with blood volume, and a single channel has no way to distinguish “brightness changed because of motion” from “brightness changed because of pulse.” Both look like the same kind of wiggle in `G_filtered(t)` alone.
- **A single channel gives the FFT nothing to cross-check against.** With only one signal, there's no way to tell whether a given frequency peak is the real pulse or a motion artifact that happened to have periodic-ish energy in the 0.7-4 Hz band (a subject shifting position every second or two, say, or breathing-linked movement). The FFT will happily report whichever peak is largest, real or not.
- **Confirmed on this project's own test subjects (see Verification section):** on subject 6-gt, green-only FFT reports **65.7 bpm** against a ground-truth average of **82.6 bpm** — off by 17 bpm, a clinically large error — while CHROM and POS both land within about half a beat per minute of the same ground truth on that same subject. The green-only spectrum for that subject (see the sanity PNGs in `results/figures/`) is visibly scattered, with no single clean dominant peak, exactly the failure mode described above.

Common Beginner Mistake

Common beginner mistake. Assuming that because green is “the best single channel” for rPPG, using only green must be nearly as good as combining all three. In practice, combining channels isn't about finding a *better single channel* — it's about using the redundancy across channels to actively cancel out shared, non-pulse variation (motion, lighting), something no single channel can do no matter how good it is on its own.

What CHROM and POS Physically Do Differently — the Skin-Tone-Vector Picture

Both CHROM and POS start from the same physical observation: plot a small patch of skin's average (R, G, B) color as a point moving through 3D color space, frame by frame. Two very different things move that point around:

1. **Motion and lighting changes** move the point mostly along a single direction — informally, the “skin-tone vector.” Whether the room got slightly brighter, or the subject turned slightly

toward the light, the *relative proportions* of R, G, B tend to hold roughly constant even as the point moves — it’s mostly getting brighter or dimmer along a fixed direction, not changing color balance.

2. **The cardiac pulse** moves the point in a genuinely different direction, because arterial blood volume changes affect red, green, and blue light absorption by different physical amounts (a consequence of hemoglobin’s specific absorption spectrum). This pulse-driven motion is small compared to (1), but it points a different way.

Intuition First

Intuition first. Imagine a marble sitting on a slightly tilted glass table (representing the skin-tone plane that lighting/motion mostly confines its movement to), that also has a tiny motor underneath giving it a small, rapid, repeating nudge in a direction that ISN’T flat along the table’s tilt (the pulse). If you only watched the marble’s shadow along one wall (one color channel), you’d see a mix of the big slow sliding motion and the tiny rapid nudge, tangled together, hard to separate. If instead you set up your camera to watch specifically the direction *perpendicular* to the table’s tilt, the big sliding motion mostly disappears from view (it’s confined to the tilted plane) and the tiny rapid nudge becomes much easier to see clearly.

CHROM builds two fixed linear combinations of the normalized channels ($Xs = 3Rn - 2Gn$, $Ys = 1.5Rn + Gn - 1.5Bn$) chosen so that the skin-tone/motion direction affects Xs and Ys in a highly correlated, proportional way, while the pulse affects them differently. Subtracting a rescaled Ys from Xs (rescaled by $\alpha = \text{std}(Xs)/\text{std}(Ys)$, so the subtraction doesn’t leave a lopsided residual) cancels most of the shared motion/lighting component and keeps the differential pulse-driven part.

POS ($S1 = Gn - Bn$, $S2 = Gn + Bn - 2Rn$) does the same thing with a different, more generally derived pair of directions — the published POS paper shows this specific pair spans the skin-tone-orthogonal plane more robustly across a wider range of skin tones and lighting conditions than CHROM’s fixed coefficients, at the cost of being a slightly less “physically interpretable” derivation than CHROM’s absorption-spectrum reasoning. Both algorithms are ultimately doing the same kind of thing: project onto a direction where motion/lighting mostly cancels and the pulse mostly survives, informed by two different (but related) models of where in RGB space that direction sits.

Pause and Check Yourself

Pause and check yourself. If a video were recorded under a colored light source that shifted the whole scene noticeably toward one color (say, warm yellow indoor lighting versus cool daylight), would you expect that to move the skin-color point mostly along the “skin-tone” direction, mostly along the “pulse” direction, or some mix — and what does that imply about how well CHROM/POS should handle a lighting-color change compared to green-only?

Why Re-Filtering After Combination Is Necessary

`chromCombine.m` and `posCombine.m` both take already-bandpassed (0.7-4 Hz) channels as input. It would be reasonable to assume the combined output is automatically still confined to that same band — after all, a linear combination of band-limited signals is still band-limited to the same band... **in continuous, ideal math.** In practice, this segment re-runs `bandpassClean.m` on the CHROM and POS outputs before handing them to `fftHeartRate.m`, for two concrete reasons:

1. **The normalization step divides by a raw (unfiltered) mean**, and while that mean is a

scalar (not time-varying), any residual numerical imprecision or subtle nonlinearity introduced by combining three independently-filtered signals (rather than filtering the true combined physical quantity directly) can reintroduce a small amount of energy outside the original band — small, but not exactly zero.

2. **Defense in depth, matching this project’s Segment 3 philosophy.** `fftHeartRate.m` already masks its search to 0.7-4 Hz before peak-picking (a second independent safeguard — see below) — re-filtering before that is a first safeguard, not a redundant one, because it means the actual *signal* handed downstream (not just the *search window*) is clean, which matters if anyone later wants to inspect or plot the combined pulse waveform directly rather than just its FFT peak.

Common Beginner Mistake

Common beginner mistake. Reasoning “the inputs were already bandpassed, so the output must be too” and skipping the re-filter step to save a function call. This is the kind of assumption that holds in a textbook derivation but is worth re-verifying empirically rather than trusting by default — which is exactly why this segment’s mandatory sanity PNGs plot the *actual* re-filtered spectrum, not an assumed one.

The FFT Window-Length vs. Frequency-Resolution Tradeoff, in Concrete bpm Terms

This is the same tradeoff from the Orientation Lecture’s STFT material, applied here to a single whole-clip FFT (which is really just a one-window spectral estimate — the same math, without the “sliding” part of STFT).

The rule: for a signal sampled at `frameRate` Hz with `N` samples total, the FFT’s frequency bins are spaced `frameRate / N` Hz apart. That spacing is the *finest* frequency difference the FFT can distinguish — two true frequencies closer together than that will blur into the same bin or adjacent bins, no matter how clean the signal is. A **longer** recording (more samples `N`, same frame rate) gives **finer** resolution; a **shorter** recording gives **coarser** resolution. This is a hard mathematical limit, not something better peak-picking logic can work around.

Concretely, for this dataset: UBFC-rPPG DATASET_1 clips run approximately 80-90 seconds at approximately 28.6-29.8 fps. Working through the three subjects used to verify this segment (5-gt, 6-gt, 7-gt, fs ~ 28.67 fps, `N` ~ 2360-2410 frames):

`bin spacing (Hz) = frameRate / N ~ 28.67 / 2400 ~ 0.0119 Hz`

`bin spacing (bpm) = 0.0119 * 60 ~ 0.71 bpm`

So on this dataset’s typical clip length, **the achievable heart-rate precision is roughly ± 0.7 bpm** — not the fraction-of-a-bpm precision that a raw decimal FFT output (e.g. 76.4105) visually suggests. That number is exactly which bin won the peak search, not a continuous, arbitrarily precise measurement. A useful rule of thumb: **halving the clip length roughly doubles the bpm resolution** (makes it coarser) — a 40-second clip at the same frame rate would resolve only to about ± 1.4 bpm, and a 10-second clip (closer to what a live/real-time system, or a short self-collected test clip, might use) would resolve only to roughly ± 5 -6 bpm, which starts to matter clinically.

Pause and Check Yourself

Pause and check yourself. Segment 4 reports `HR_chrom` to four decimal places in its CSV output. Given the ~ 0.7 bpm bin spacing above, is reporting that many decimal places misleading, and if so, what would be a more honest way to communicate the estimate’s actual precision to someone reading the results table?

Why CHROM and POS reported identical bpm on all three test subjects. This was checked directly (not assumed): the two combined signals are similar but not literally identical (verified by comparing them sample-by-sample), just strongly correlated, since both are different linear combinations of the same three physically-correlated color channels, both tuned to isolate the same underlying cardiac frequency. Given the ~ 0.7 bpm bin spacing above, two signals whose true dominant frequency both fall inside the same bin will report the exact same `hrBpm`, even though a continuous (infinite-resolution) frequency estimate would likely show a small difference between them. This is an expected consequence of the resolution ceiling, not a sign that the two algorithms produced the same computation.

Common Pitfalls in This Segment

Picking a peak outside the valid band by mistake. Covered in detail in the Line-by-Line document — `fftHeartRate.m` masks the search to 0.7-4 Hz *before* calling `max()`, specifically so a strong out-of-band spike (drift residue near 0 Hz, high-frequency noise above 4 Hz) can never be selected, even by accident. Finding the global peak first and checking afterward whether it happens to be in range is a materially weaker guarantee than masking first — always mask first.

Multiple close peaks from a noisy ROI or heavy motion. Even inside the valid band, a noisy signal can show two or three comparably-sized peaks close together (for example, a true pulse peak plus a nearby motion-linked peak that didn’t fully cancel). `fftHeartRate.m`’s `max()` will pick whichever is *numerically* largest, which is not guaranteed to be the physiologically correct one if the real peak and a noise peak are close in magnitude. This is precisely why Step 6’s sanity PNGs are mandatory rather than optional: a human glancing at the spectrum can immediately tell “clean, single, obvious peak” (like 5-gt’s CHROM/POS spectra) apart from “scattered energy, ambiguous winner” (like 6-gt’s green-only spectrum) in a way no automated check in this segment currently does. If a subject’s sanity PNG shows the latter pattern for CHROM or POS (not just green-only, which is expected to be weaker), that subject’s estimate should be treated with real skepticism before being trusted downstream.

Common Beginner Mistake

Common beginner mistake. Trusting a bpm number just because the script ran without error. `fftHeartRate.m` will always return *some* value in range — even for a spectrum with no clean peak at all, it just returns whichever noisy bin happened to be tallest. A plausible-looking bpm number is not the same as a *trustworthy* one; that judgment call is exactly what the mandatory sanity PNG is for.

How This Segment’s Three Estimates Feed Into Segment 6

For every subject, this segment produces three independent bpm estimates — `HR_chrom`, `HR_pos`, `HR_green` — saved to `data/processed/<subjectID>_hr_estimates.mat` and appended as one row

to `results/metrics/segment4_hr_summary.csv`, alongside whatever ground-truth HR is available for that subject (NaN if not — see the Team README for exactly when that happens).

This is deliberately **not** a formal validation step. The `abs_error_*` columns in the CSV are for quick, per-subject eyeballing only — “does this number look roughly sane” — not a substitute for Segment 6’s actual job: `validation/runLOS0.m` and `validation/computeMetrics.m` (both still untouched stubs, out of scope for this segment) will run proper leave-one-subject-out cross-validation across every available subject and report MAE, RMSE, Pearson correlation, and Bland-Altman agreement for each of the three HR estimation methods this segment provides — letting the team make an evidence-based call on which of CHROM, POS, or green-only (or some combination) to carry forward, instead of assuming one is best from the literature alone.

Verification on This Project’s Own Data

This segment was run end-to-end on the three DATASET_1 subjects with both Segment 2 and Segment 3 output already available (5-gt, 6-gt, 7-gt). `io/loadGroundTruth.m` is still an unimplemented stub as of Segment 4 (it belongs to a later segment and was intentionally left untouched), so the batch script’s own ground-truth column reads NaN for all three subjects by design. As an independent manual check (outside the pipeline code, just reading `gtdump.xmp`’s HR column directly), the mean ground-truth HR for these three subjects is:

Subject	HR_chrom	HR_pos	HR_green	Ground truth (mean, manual check)
5-gt	76.41 bpm	76.41 bpm	72.84 bpm	77.32 bpm
6-gt	83.02 bpm	83.02 bpm	65.69 bpm	82.57 bpm
7-gt	91.77 bpm	91.77 bpm	95.41 bpm	94.68 bpm

CHROM and POS land within about 1-3 bpm of ground truth on all three subjects. Green-only is close on two subjects but off by nearly 17 bpm on 6-gt — exactly the motion-fragility failure mode this document opened with, and visibly confirmed by that subject’s noisy, ambiguous green-only spectrum in `results/figures/6-gt_heartrate_sanity.png`.